Build & Deploy

End-to-end guide for getting the Automated Thematic Analysis stack running, configured, and verified. Synthesises the existing per-service READMEs:

- Root README — high-level overview and bootstrap scripts
- Backend/README.md — FastAPI service detail
- Frontend/README.md — Flask UI detail

Architecture at a glance

Three services are defined in `docker-compose.yml` at the repository root:

Service	Image / target	Host port	Purpose
`db`	`postgres:16-alpine`	`5433`	PostgreSQL 16 with named volume `pgdata`
`api`	`Backend/Dockerfile` → `runtime`	`8000` (`APP_PORT`)	FastAPI backend, async SQLAlchemy, LangChain
`frontend`	`Frontend/Dockerfile` → `runtime`	`3000` (`FRONTEND_PORT`)	Flask + Jinja UI

Plus two test-only services (`api-test` , `frontend-test`) gated behind the `test` Docker Compose profile.

The frontend talks to the backend over the internal Docker network at `http://api:8000/api/v1` (set on the `frontend` service). The backend reads its DB URL from `Backend/.env` but Compose overrides it to point at `db:5432` inside the network, so a single `Backend/.env` works for both local and containerised runs.

Prerequisites

Required:

- **Docker Engine** (with Compose v2 — verify with `docker compose version`)

Optional, only needed for native development without Docker:

- **Python 3.11+**
- uv (recommended) or `pip`
- A local PostgreSQL 16 instance if you don't want to run the `db` container

An **LLM API key** is required for codebook generation features. Either:

- **NHR@FAU** gateway — request at https://hpc.fau.de/request-llm-api-key/ (default)
- **GWDG Academic Cloud** — alternative provider

You only need one of the two.

Quick deploy (recommended)

From the repository root:

Linux / macOS / Git Bash on Windows:

```
chmod +x setup.sh
./setup.sh
```

Windows PowerShell:

```
.\setup.ps1
```

What the script does:

1. Verifies Docker Engine and Compose v2 are installed and the daemon is running.

2. Creates `Backend/.env` from `Backend/.env.example` if it doesn't yet exist.
3. Runs `docker compose build` then `docker compose up -d`.
4. Polls the API's `/api/v1/health/ready` endpoint until it returns 200, then prints the service URLs.

One mandatory follow-up: open `Backend/.env` and set your LLM API key. The bootstrap leaves the placeholder values (`<your_nhr_fau_key_here>` etc.) intentionally so the stack starts even before you've requested a key. Until you fill it in, codebook generation will fail with a clear error in the job runner.

Common bootstrap-script options (run with `--help` for the full list):

Task	Linux / macOS	Windows PowerShell
Start stack (detached)	`./setup.sh`	`.\setup.ps1`
Start with foreground logs	`./setup.sh -f`	`.\setup.ps1 -Foreground`
Run the test suites	`./setup.sh --test`	`.\setup.ps1 -Test`
Stop the stack	`./setup.sh --down`	`.\setup.ps1 -Down`
Stop and **wipe the DB volume**	`./setup.sh --down-volumes`	`.\setup.ps1 -DownVolumes`

Manual deploy

If you'd rather drive Docker Compose directly:

```
# 1. Configure
cp Backend/.env.example Backend/.env
# Edit Backend/.env — at minimum set LLM_API_KEY_FAU (or LLM_API_KEY for Academic Cloud)

# 2. Build + start
docker compose up --build -d

# 3. Wait for the api container to report healthy
docker compose ps

# 4. Tail logs if anything looks off
docker compose logs api --tail=50 -f
```

Once `db`, `api`, and `frontend` are all healthy, the URLs are:

Surface	URL
Frontend UI	http://localhost:3000
Backend API	http://localhost:8000
API docs (Swagger)	http://localhost:8000/docs
Health check (ready)	http://localhost:8000/api/v1/health/ready

Configuration

All configuration is environment-driven via `Backend/.env` (the frontend reads `BACKEND_API_URL` from its Compose environment block; see docker-compose.yml).

Backend — `Backend/.env`

The full list of variables is documented in `Backend/.env.example`. The most important ones:

Variable	Required	Default
`LLM_API_KEY_FAU`	yes (if `SELECTED_API=FAU`)	—
`LLM_API_KEY`	yes (if `SELECTED_API=ACADEMIC`)	—
`SELECTED_API`	no	`FAU`
`LLM_MODEL_FAU`	no	`gpt-oss-120b`

Variable	Required	Default
`LLM_MODEL`	no	`gemma-3-27b-it`
`LLM_REQUEST_TIMEOUT_S`	no	`120.0`
`DATABASE_URL`	no inside Docker	`postgresql+asyncpg://postgres:postgres@localh`
`CORS_ALLOWED_ORIGINS`	no	`["http://localhost:3000"]`
`APP_PORT`	no	`8000`
`LOG_LEVEL`	no	`INFO`

Frontend (set in Compose, not in a `.env`)

Variable	Default	Notes
`BACKEND_API_URL`	`http://api:8000/api/v1`	Internal Docker DNS — don't change for in-Compose deploys
`BACKEND_TIMEOUT_S`	`60.0`	HTTP client timeout
`APP_ENV`	`development`	Set to `production` for prod deploys
`SECRET_KEY`	`dev-secret`	**Change for production.**
`MAX_UPLOAD_SIZE_MB`	`10`	Per-file upload cap
`FRONTEND_PORT`	`3000`	Host port

Verification

After the stack starts, run through these checks in order. Each fails fast and points to a specific service if something's wrong.

1. **Containers up + healthy.**

   ```
   docker compose ps
   ```

   `db`, `api`, `frontend` should all read `Up`. The `api` row reports its health check status; wait for `Healthy`.
2. **Backend readiness.**

   ```
   curl -fsS http://localhost:8000/api/v1/health/ready
   ```

   Returns `{"success": true, "data": {"status": "ready"}, ...}`. A 502 means `api` hasn't finished startup; a connection refused means port mapping is off.
3. **Database schema initialised.**

   ```
   docker compose logs api | grep -E "schema (initialization|verification)"
   ```

   Look for `Database schema initialization completed`. If you see `Missing tables: ...` you have a schema-drift issue — wipe the volume (see Troubleshooting).
4. **LLM connectivity.** From the `api` container:

   ```
   docker compose cp Backend/scripts/test_nhr_fau_api.py api:/app/test_nhr_fau_api.py
   docker compose exec api python test_nhr_fau_api.py
   ```

Should print ✅ `API call succeeded!` plus a short generated sentence. If it fails:

- `No API key set for SELECTED_API='FAU'` → key missing in `Backend/.env`; edit and `docker compose restart api`.
- Connection / timeout → check VPN or campus-only network restrictions.
- 401 / 403 → key invalid or expired.

5. **Frontend reachable.**

```
curl -fsS http://localhost:3000/health
```

Returns `OK`. Open http://localhost:3000 in a browser to confirm the UI renders.

6. **End-to-end smoke (UI).** From the home page:
7. **Upload** → drop a small `.txt`, `.jsonl`, `.docx`, or `.pdf` interview file (one of the samples under `Backend/tests/test-data/` works).
8. **Codebooks** → "Create New Codebook" → "Fully automatic" → name → Confirm.
9. Progress page should tick from `queued` → `running` → `succeeded` and redirect to the codebook list with the new codebook visible.

Iteration

While developing, you usually want to rebuild only one service:

```
# Backend only
docker compose build api && docker compose up -d --no-deps api

# Frontend only
docker compose build frontend && docker compose up -d --no-deps frontend
```

Compose's `develop.watch` is configured for both services to **sync source changes** (`Backend/app/` and `Frontend/web/`) into the running containers — no rebuild needed for code changes once `docker compose watch` is running.

Tests

Both services have their own test suites; both run inside Docker via the `test` profile.

```
# Run all tests (both backend and frontend), backed by the live db container
docker compose --profile test up -d db
docker compose --profile test run --rm api-test       # backend
docker compose --profile test run --rm frontend-test  # frontend
```

Or via the bootstrap script:

```
./setup.sh --test
```

Backend runs `pytest` with coverage; the HTML report is written to `Backend/htmlcov/` on the host via volume mount.

Frontend uses a `FakeBackend` fixture so no live backend is required for the test container.

Stack lifecycle

```
# Stop the stack but keep containers + volumes
docker compose stop

# Stop and remove containers (DB volume preserved)
docker compose down

# Stop, remove containers AND wipe the DB volume (loses all uploaded transcripts and
codebooks)
docker compose down -v
```

Production considerations

The default Compose file is set up for **local development**: hot reload enabled (`--reload` on uvicorn, `FLASK_DEBUG=1`), default secrets, the API and DB ports exposed on the host. For a production-shaped deploy:

1. **Disable hot reload** by overriding the `api` service `command` to drop `--reload` , and set `FLASK_DEBUG=0` , `APP_ENV=production` on the `frontend` service.
2. **Set a real** `SECRET_KEY` on the frontend (used to sign Flask sessions).
3. **Don't expose the DB port** (`5433`) externally — drop the `db.ports` mapping.
4. **Pin LLM and DB credentials** via your platform's secret store rather than `Backend/.env` on disk.
5. **Build with** `--no-cache` for repeatable images, and tag explicitly (e.g. `docker build -t ata-api:v1.0 ./Backend`) instead of letting Compose name them.
6. **Run migrations / schema bootstrap intentionally.** The backend currently uses `Base.metadata.create_all` at startup (Backend/app/database.py) which only creates *missing* tables — it never alters existing ones. Treat schema changes as a destructive operation (`docker compose down -v`) until Alembic is added. See Troubleshooting for the failure mode this causes.
7. **Persist** `pgdata` to a managed volume (e.g. mounted cloud disk) rather than the Docker-managed `pgdata` volume.

Troubleshooting

Schema drift after pulling new commits

Symptom: any request that touches a recently-added column fails with `UndefinedColumnError` , e.g. `column "demographic_row_id" of relation "corpus_documents" does not exist` .

Cause: The backend uses `Base.metadata.create_all` at startup. It only creates *missing tables*, never adds new columns to existing ones. If your local `pgdata` volume was created before the schema change, the column will never appear.

Fix:

```
docker compose down -v        # destroys the pgdata volume
docker compose up --build
```

You lose any local data; reupload your transcripts.

"Backend unreachable" in the UI

Either the `api` container isn't healthy yet (see Verification step 1), or the `frontend` service has the wrong `BACKEND_API_URL` . Inside the Compose network it must be `http://api:8000/api/v1` (the service name, not `localhost`).

Cache cleanup if the UI looks stale

Walk down this table from cheap to nuclear. From Frontend/README.md:

#	Layer	Fix
1	Browser cache	Hard refresh: `Ctrl + Shift + R` (or Incognito)
2	Frontend image	`docker compose build --no-cache frontend && docker compose up -d --no-deps frontend`
3	Docker BuildKit cache	`docker builder prune -af` , then redo step 2
4	Python bytecode (local dev only)	`find . -name __pycache__ -type d -exec rm -rf {} +`
5	Full reset (keeps DB volume)	`docker compose down && docker builder prune -af && docker compose build --no-cache && docker compose up -d`

Avoid `docker system prune -af --volumes` unless you actually want a blank database too — it drops `pgdata` .

Codebook generation fails with `ForeignKeyViolationError`

Symptom: the codebook generation job moves from `running` to `failed` , with `error_message` along the lines of `insert or update on table "codes" violates foreign key constraint "codes_codebook_id_fkey"` .

Cause: SQLAlchemy unit-of-work autoflush ordering can interleave parent and child INSERTs incorrectly when the model graph lacks explicit `relationship()` declarations. The persistence path in `Backend/app/services/codebook_generation.py` uses a layered-flush pattern (`session.flush()` between each dependency layer) to work around this. If you see this error, confirm the patched version is what's deployed; recent versions on `main` include the fix.

Port already in use

The compose file falls back to defaults `8000` (API) and `3000` (frontend). To override without editing files:

```
APP_PORT=8001 FRONTEND_PORT=3001 docker compose up -d
```

Where to look next

- Backend README — project structure, response envelope, in-Docker test runs
- Frontend README — page-by-page routes, error-handling model, Dockerfile build stages
- `Documentation/ingestion-pipeline.md` — corpus / document / chunk data model and ingest API
- `Documentation/codebook-generation.md` — sync vs. async generation endpoints, job lifecycle
- `Documentation/LLM-cluster-documentation.md` — FAU GPU cluster vs. Academic Cloud notes
- `Documentation/csv-codebook-standard.md` — uploadable codebook CSV format